

Software Lifecycle Models

A software lifecycle model is a standardised format for

- planning
- organising, and
- running

a new development project.

Hundreds of different kinds of models are known and used.

Many are minor variations on just a small number of basic models. In this section we:

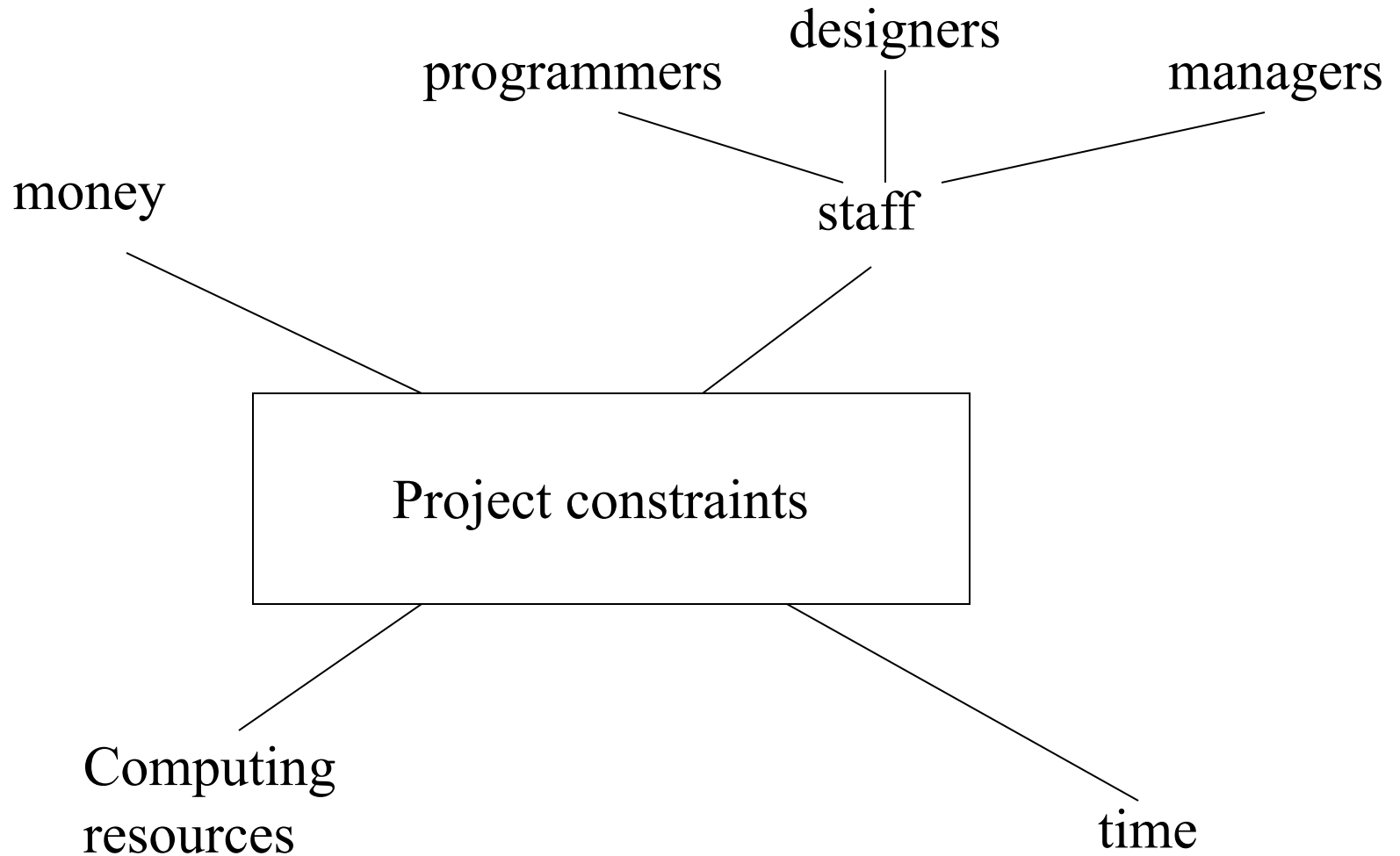
- **survey** the main types of model, and
- consider **how to choose** between them.

Planning with Models

SE projects usually live with a **fixed financial budget**. (An exception is maintenance?)

Additionally, time-to-market places a strong **time constraint**.

There will be other **project constraints** such as staff.



Examples of Project Constraints

Project planning is the art of scheduling the **necessary activities**, in time, space and across staff in order to optimise:

- project risk [low] (see later)
- profit [high]
- customer satisfaction [high]
- worker satisfaction [high]
- long-term company goals

Questions:

1. What are these necessary activities?
(besides programming)
2. Are there good patterns of organisation
that we could copy?

A project plan contains much information,
but must at least describe:

- resources needed
(people, money, equipment, etc)
- dependency & timing of work
(flow graph, work packages)
- rate of delivery *(reports, code, etc)*

It is impossible to measure rate of progress
except with reference to a plan.

In addition to project members, the following may need access to parts of the project plan:

- Management,
- Customers
- Subcontractors
- Suppliers
- Investors
- Banks

Project Visibility

Unlike other engineers
(e.g. civil, electronic, chemical ... etc.)
software engineers do not produce anything
physical.

It is inherently difficult to monitor an SE
project due to **lack of visibility**.

This means that SE projects must produce
additional deliverables (*artifacts*)

which are visible, such as:

- *Design documents/ prototypes*
- *Reports*
- *Project/status meetings*
- *Client surveys (e.g. satisfaction level)*

What is a Lifecycle Model?

Definition.

A (software/system) *lifecycle model* is a description of the sequence of activities carried out in an SE project, and the relative order of these activities.

It provides a fixed **generic framework** that can be tailored to a specific project.

Project specific **parameters** will include:

- Size, (person-years)
- Budget,
- Duration.

**project plan =
lifecycle model + project parameters**

There are hundreds of different lifecycle models to choose from, e.g:

- *waterfall,*
- *code-and-fix*
- *spiral*
- *rapid prototyping*
- *unified process (UP)*
- *agile methods, extreme programming (XP)*
- *COTS ...*

but many are minor variations on a smaller number of basic models.

By changing the lifecycle model, we can **improve and/or tradeoff**:

- *Development speed (time to market)*
- *Product quality*
- *Project visibility*
- *Administrative overhead*
- *Risk exposure*
- *Customer relations, etc, etc.*

Normally, a lifecycle model covers the entire **lifetime of a product.**

From *birth of a commercial idea*
to *final de-installation of last release*

i.e. The three main phases:

- *design,*
- *build,*
- *maintain.*

Note that we can sometimes **combine**
lifecycle models,

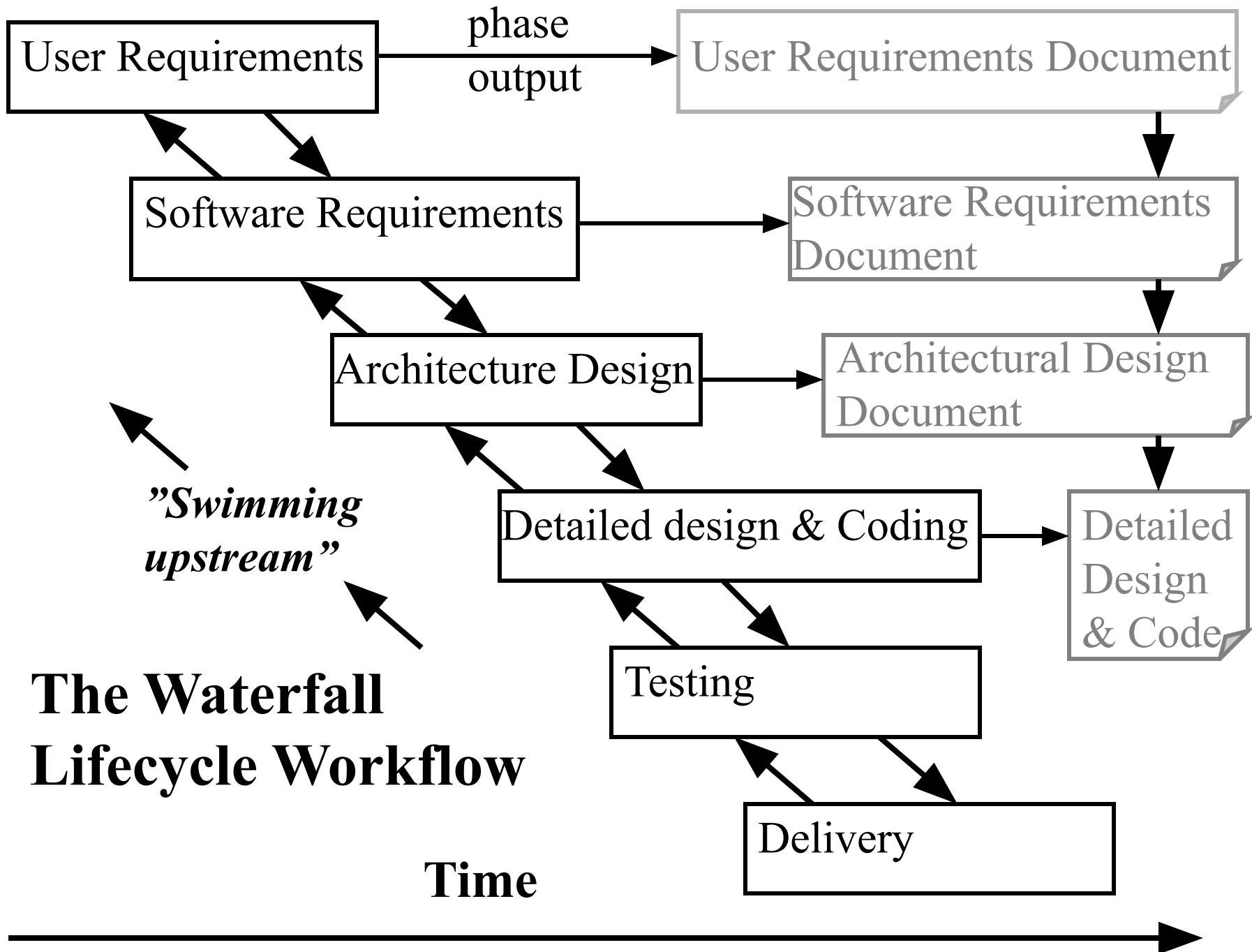
e.g. waterfall inside evolutionary – onboard
shuttle software

We can also **change** lifecycle model between
releases as a product matures,

e.g. rapid prototyping → waterfall

The Waterfall Model

- The waterfall model is the classic lifecycle model – it is widely known, understood and (commonly?) used.
- In some respect, waterfall is the ”common sense” approach.
- Introduced by Royce 1970.



Advantages

1. Easy to understand and implement.
2. Widely used and known (in theory!)
3. Reinforces good habits: define-before- design, design-before-code
4. Identifies deliverables and milestones
5. Document driven, URD, SRD, ... etc. Published documentation standards, e.g. PSS-05.
6. Works well on mature products and weak teams.

Disadvantages I

1. Idealised, doesn't match reality well.
2. Doesn't reflect iterative nature of exploratory development.
3. Unrealistic to expect accurate requirements so early in project
4. Software is delivered late in project, delays discovery of serious errors.

Disadvantages II

5. Difficult to integrate risk management
6. Difficult and expensive to make changes to documents, "swimming upstream".
7. Significant administrative overhead, costly for small teams and projects.

Code-and-Fix

This model starts with an informal general product idea and just develops code until a product is "ready" (or money or time runs out). Work is in random order.

Corresponds with no plan! (**Hacking!**)

Advantages

1. No administrative overhead
2. Signs of progress (code) early.
3. Low expertise, anyone can use it!
4. Useful for small “*proof of concept*” projects, e.g. as part of risk reduction.

Disadvantages

1. Dangerous!
 1. No visibility/control
 2. No resource planning
 3. No deadlines
 4. Mistakes hard to detect/correct
2. Impossible for large projects, communication breakdown, chaos.

Spiral Model

Since end-user requirements are hard to obtain/define, it is natural to develop software in an *experimental* way: e.g.

1. Build some software
2. See if it meets customer requirements
3. If no goto 1 else stop.

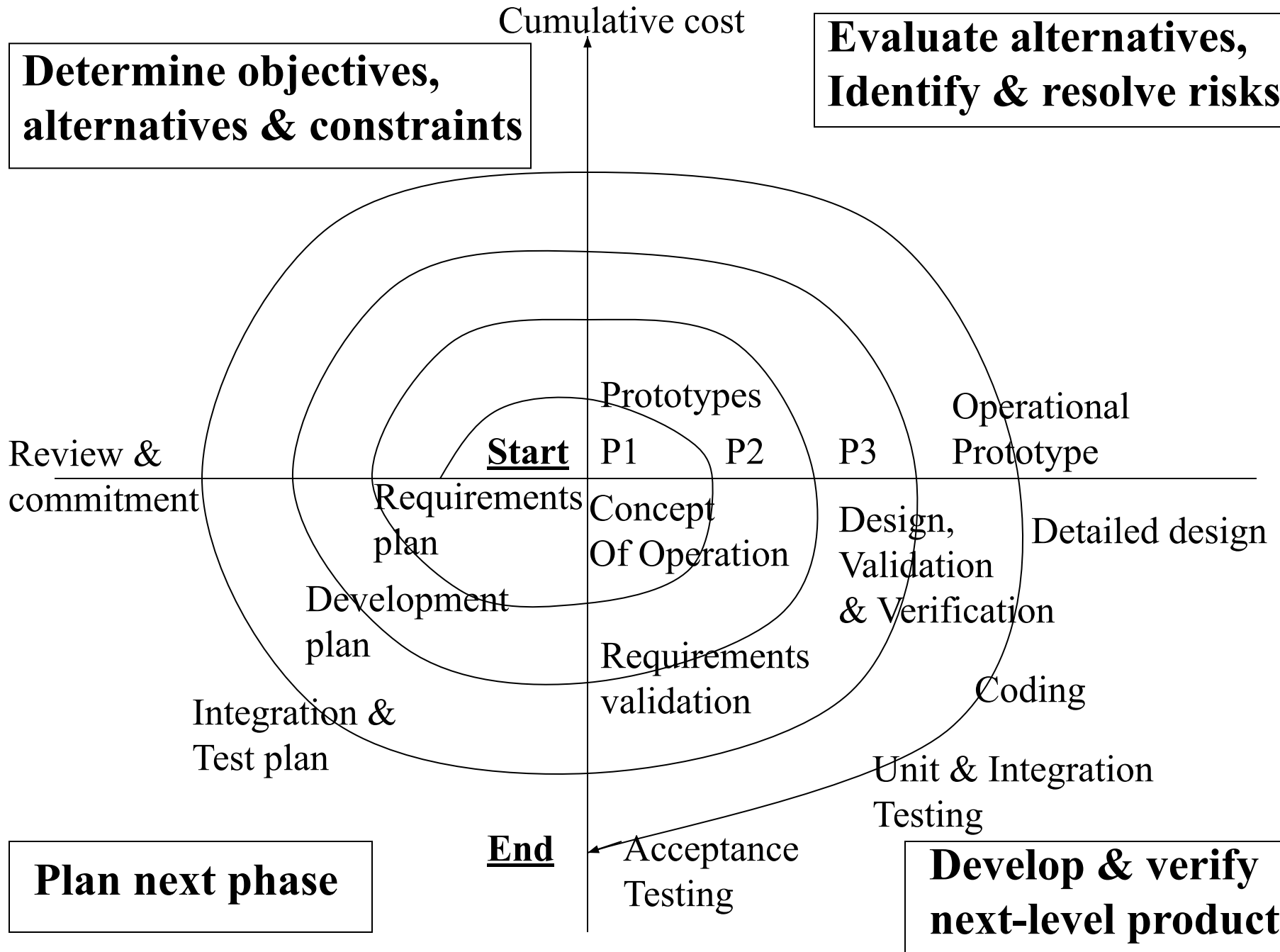
This loop approach gives rise to structured **iterative lifecycle models.**

In 1988 Boehm developed the spiral model as an iterative model which includes *risk analysis* and *risk management*.

Key idea: on each iteration identify and solve the sub-problems with the *highest risk*.

**Determine objectives,
alternatives & constraints**

**Evaluate alternatives,
Identify & resolve risks**



Each cycle follows a waterfall model by:

1. Determining objectives
2. Specifying constraints
3. Generating alternatives
4. Identifying risks
5. Resolving risks
6. Developing next-level product
7. Planning next cycle

Advantages

1. Realism: the model accurately reflects the iterative nature of software development on projects with unclear requirements
2. Flexible: incorporates the advantages of the waterfall and rapid prototyping methods
3. Comprehensive model decreases risk
4. Good project visibility.

Disadvantages

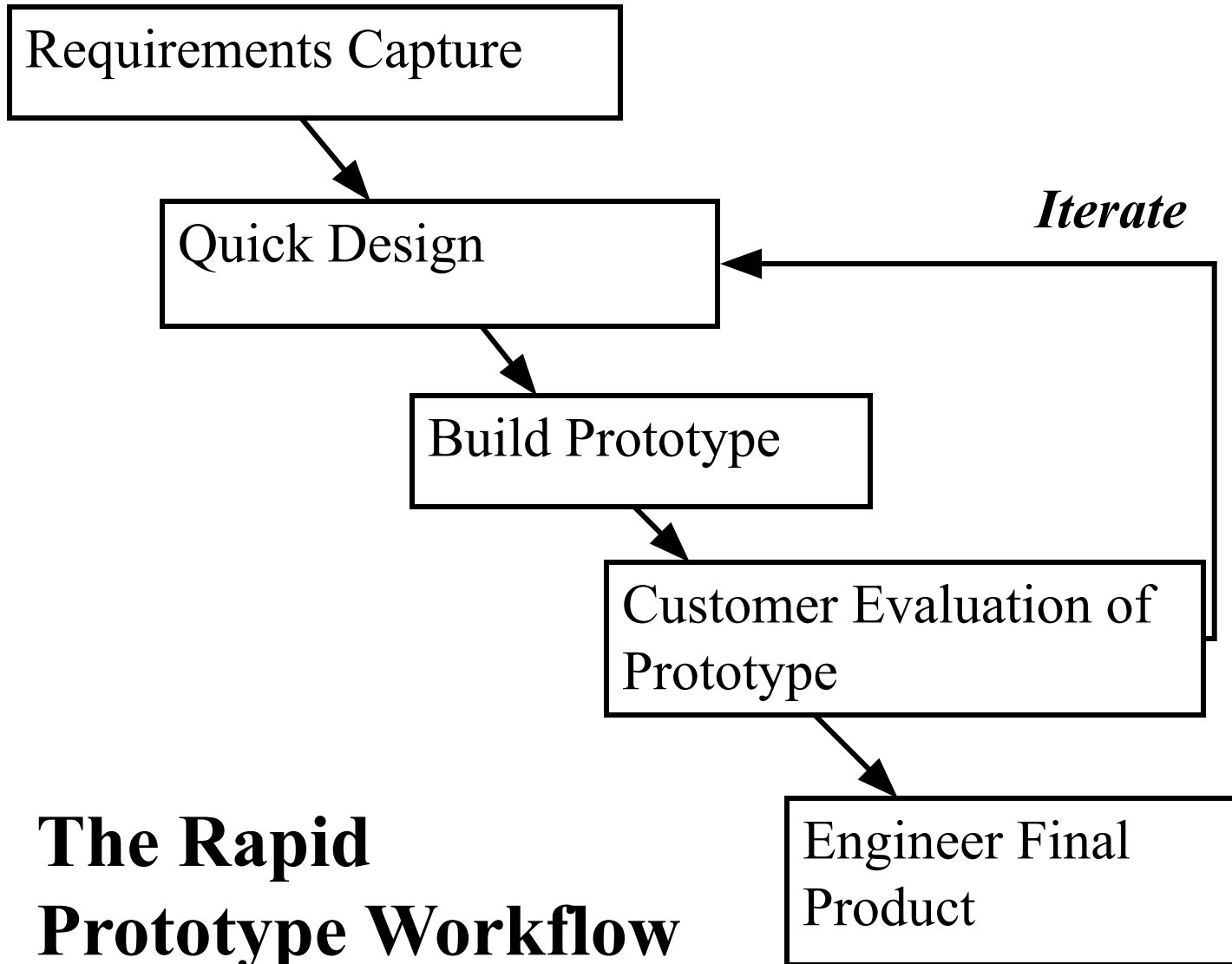
- Needs technical expertise in risk analysis to really work
- Model is poorly understood by non-technical management, hence not so widely used
- Complicated model, needs competent professional management. High administrative overhead.

Rapid Prototyping

Key idea: Customers are non-technical and usually don't know what they want/can have.

Rapid prototyping emphasises requirements analysis and validation, also called:

- *customer oriented development,*
- *evolutionary prototyping*



The Rapid Prototype Workflow

Advantages

1. Reduces risk of incorrect user requirements
2. Good where requirements are changing/uncommitted
3. Regular visible progress aids management
4. Supports early product marketing

Disadvantages I

1. An unstable/badly implemented prototype often becomes the final product.
2. Requires extensive customer collaboration
 - Costs customers money
 - Needs committed customers
 - Difficult to finish if customer withdraws
 - May be too customer specific, no broad market

Disadvantages II

3. Difficult to know how long project will last
4. Easy to fall back into code-and-fix without proper requirements analysis, design, customer evaluation and feedback.

Agile (XP) Manifesto

XP = Extreme Programming emphasises:

- Individuals and interactions
 - **Over processes and tools**
- Working software
 - **Over documentation**
- Customer collaboration
 - **Over contract negotiation**
- Responding to change
 - **Over following a plan**

Agile Principles (Summary)

- Continuous delivery of software
- Continuous collaboration with customer
- Continuous update according to changes
- Value participants and their interaction
- Simplicity in code, satisfy the spec

XP Practices (Summary)

- Programming in pairs
- Test driven development
- Continuous planning, change , delivery
- Shared project metaphors, coding standards and ownership of code
- No overtime! (Yeah right!)

Advantages

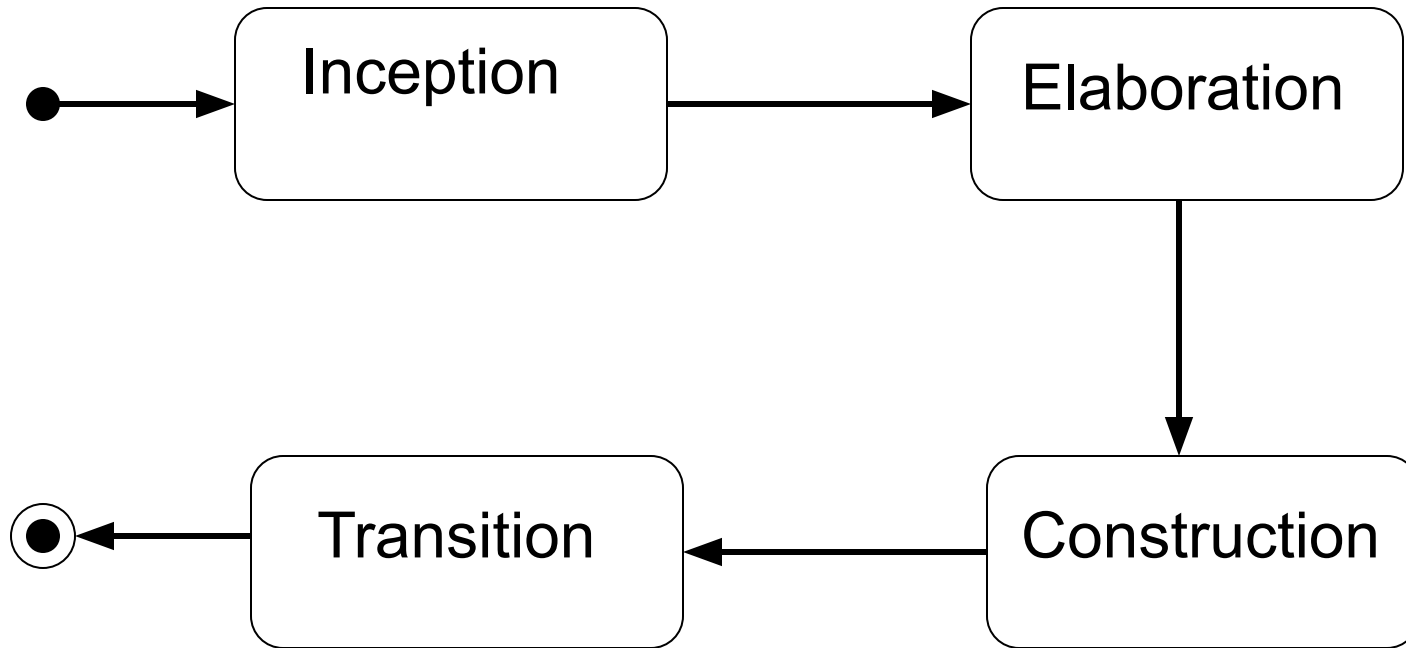
- Lightweight methods suit small-medium size projects
- Produces good team cohesion
- Emphasises final product
- Iterative
- Test based approach to requirements and quality assurance

Disadvantages

- Difficult to scale up to large projects where documentation is essential
- Needs experience and skill if not to degenerate into code-and-fix
- Programming pairs is costly
- Test case construction is a difficult and specialised skill.

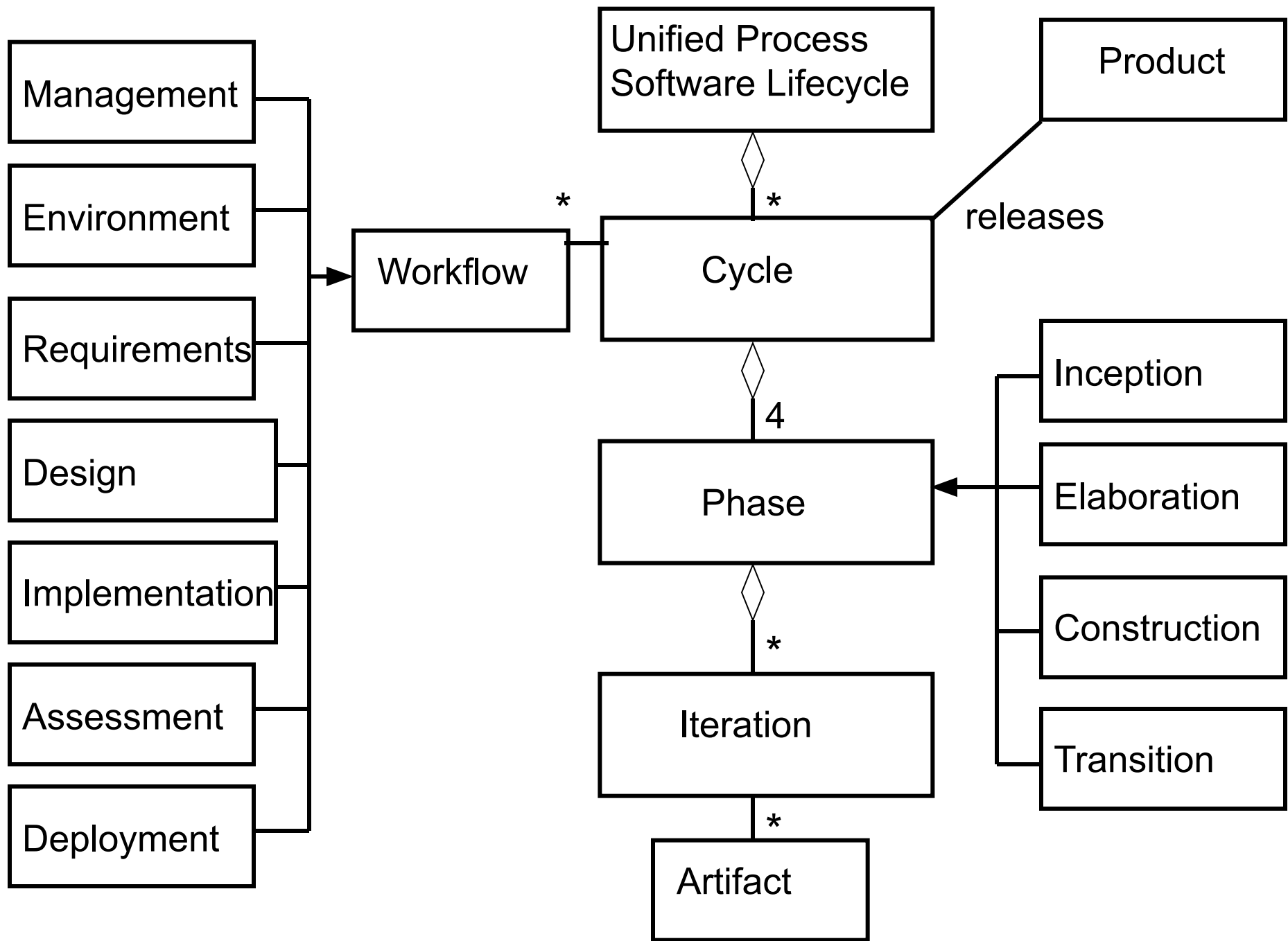
Unified Process (UP)

- Booch, Jacobson, Rumbaugh 1999.
- Lifetime of a software product in **cycles**:
- ***Birth, childhood, adulthood, old-age, death.***
- Product maturity stages
- Each cycle has phases, culminating in a new release (c.f. Spiral model)

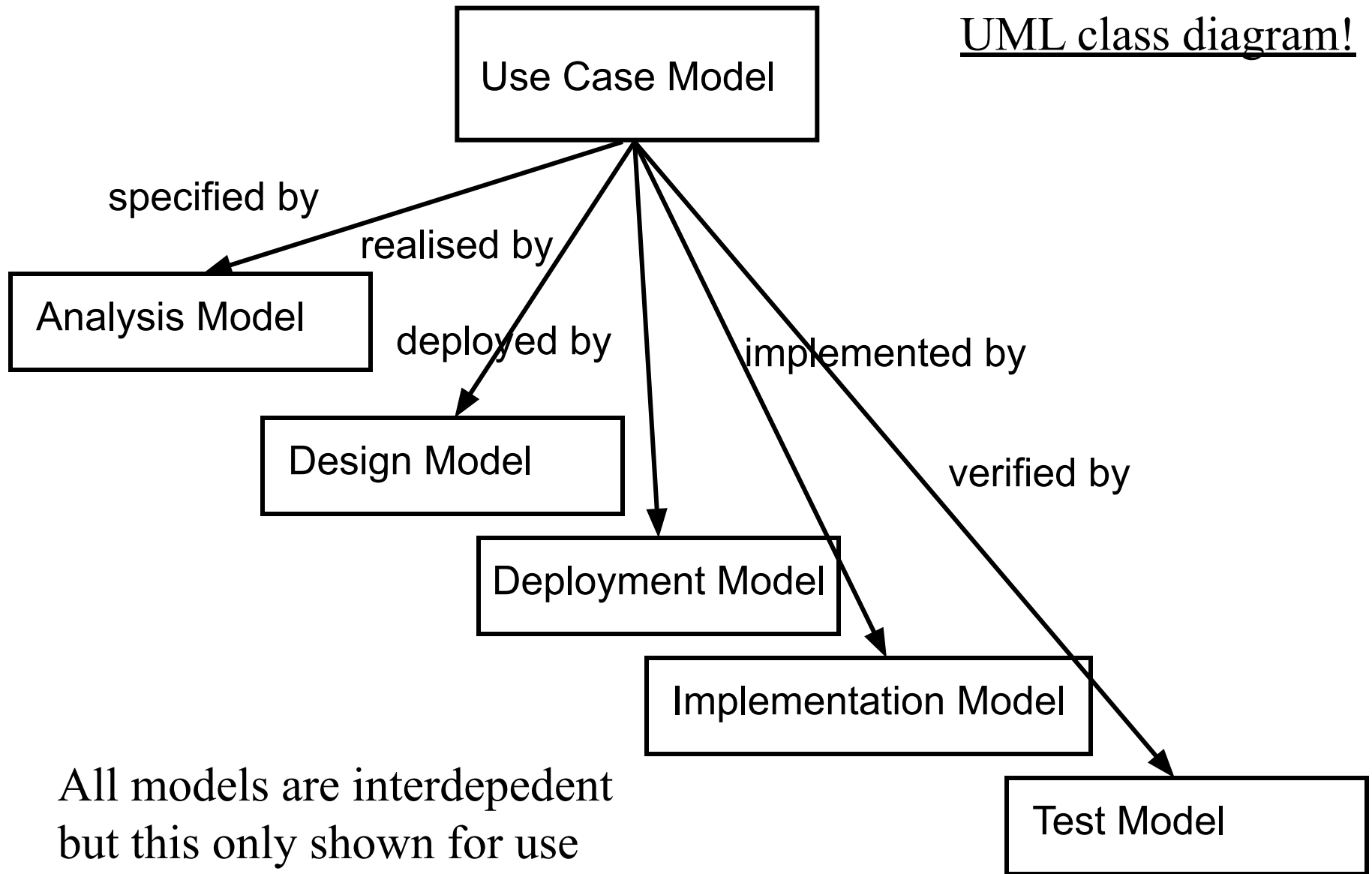


**UP Lifecycle – single phase workflow
(drawn as a UML Statechart!)**

- **Inception** – identify core use cases, and use to make architecture and design tradeoffs. Estimate and schedule project from derived knowledge.
- **Elaboration** – capture detailed user requirements. Make detailed design, decide on build vs. buy.
- **Construction** – components are bought or built, and integrated.
- **Transition** – release a mature version that satisfies acceptance criteria.



UML class diagram!



All models are interdependent
but this only shown for use
case model

COTS

- **COTS =**
Commercial Off-The-Shelf software
- Engineer together a solution from existing commercial software packages using minimal software *"glue"*.
- E.g. using databases, spread sheets, word proccessors, graphics software, web browsers, etc.

Advantages

- Fast, cheap solution
- May give all the basic functionality
- Well defined project, easy to run

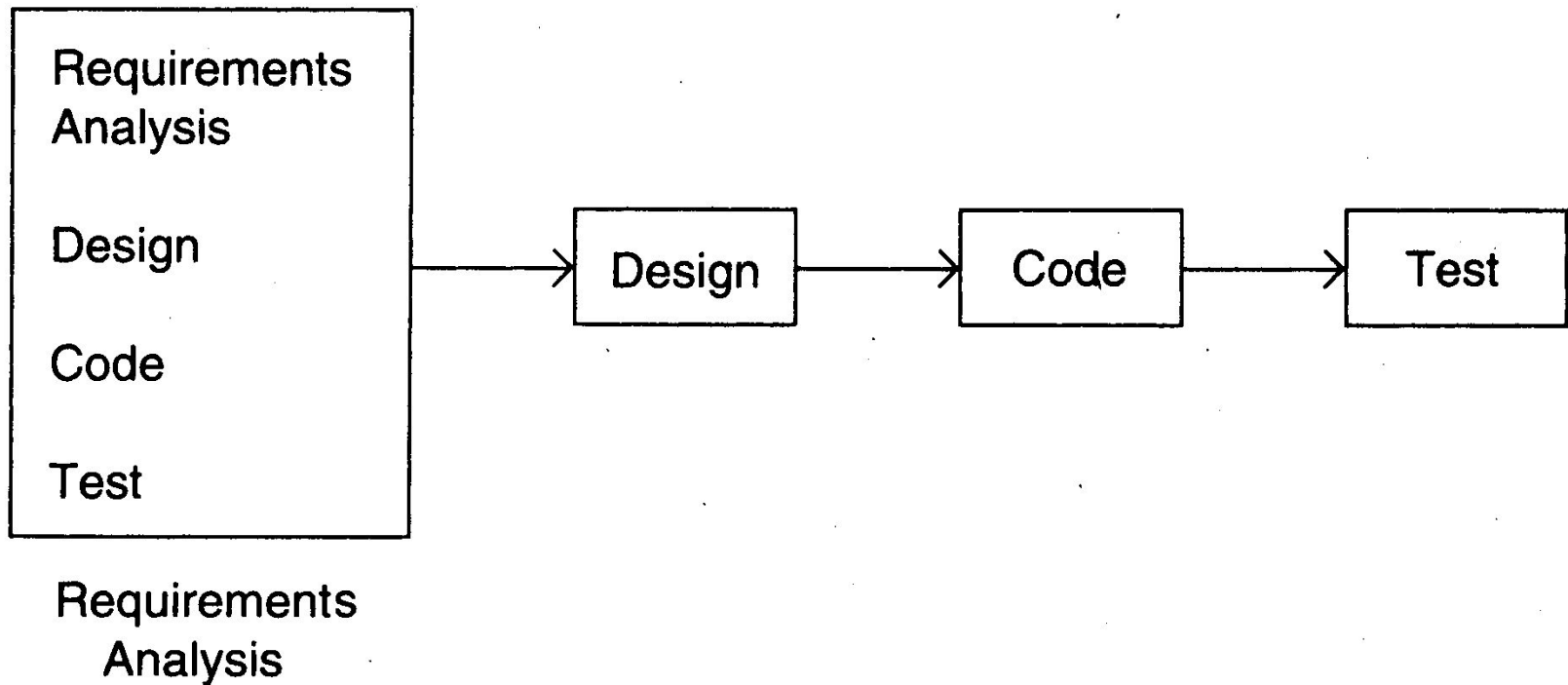
Disadvantages

- Limited functionality
- Licensing problems, freeware, shareware, etc.
- License fees, maintainance fees, upgrade compatibility problems

Prototyping

- Addresses the requirement specification limitation of waterfall
- Instead of freezing requirements only by discussions, a prototype is built to understand the requirements
- Helps alleviate the requirements risk
- A small waterfall model replaces the requirements stage

Prototyping



Development of prototype

- Starts with initial requirements
- Only key features which need better understanding are included in prototype
- No point in including those features that are well understood
- Feedback from users taken to improve the understanding of the requirements

Prototyping

- Cost can be kept low
 - Build only features needing clarification
 - “quick and dirty” – quality not important, scripting etc can be used
 - Things like exception handling, recovery, standards are omitted
 - Cost can be a few % of the total
 - Learning in prototype building will help in building, besides improved requirements

Prototyping

- Advantages

- Requirement will be more stable and more likely to satisfy user needs
- Early opportunity to explore scale/performance issues
- Ability to modify or cancel the project early
- Enhanced user engagement

- Disadvantages:

- Potential hit on cost and schedule
- Potential false sense of security if prototype does not focus on key (high risk) issues

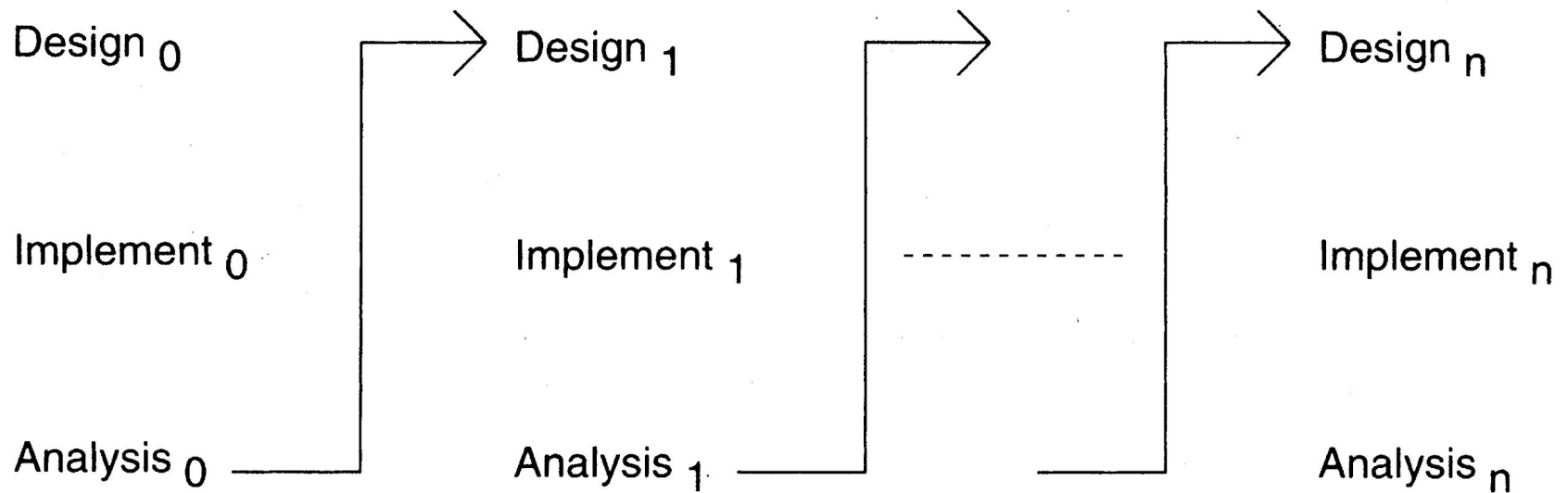
Prototyping

- Applicability:
 - When req are hard to elicit
 - When confidence in reqs is low
 - Where reqs are not well understood
 - When design is driven by user needs
- Variants
 - Paper Prototypes
 - UI Prototypes
 - Technology Proving
 - Rapid Prototyping environments

Iterative Development

- Counters the “all or nothing” drawback of the waterfall model
- Combines benefit of prototyping and waterfall
- Develop and deliver software in increments
- Each increment is complete in itself
- Can be viewed as a sequence of waterfalls
- Feedback from one iteration is used in the future iterations

Iterative Enhancement



Iterative Development

- Most Software Products follow it
- Used commonly in customized development also
 - Businesses want quick response for sw
 - Cannot afford the risk of all-or-nothing
- Newer approaches like XP, Agile,... all rely on iterative development

Iterative Development

- Benefits
 - Get-as-you-pay
 - feedback for improvement
- Drawbacks
 - Architecture/design may not be optimal
 - Amount of refactoring may increase
 - Total cost may increase

Iterative Development

- Applicability
 - where response time is important,
 - risk of long projects cannot be taken,
 - all req not known
- Execution
 - Each iteration is a mini waterfall – decide the specs, then plan the iteration
 - Length of iteration driven by amount of new functionality to be added in an iteration

Timeboxing

- Time boxing is like Iterative development but
 - fix an iteration duration, then determine the specs
- Divide iteration in a few equal stages
- Use pipelining concepts to execute iterations in parallel

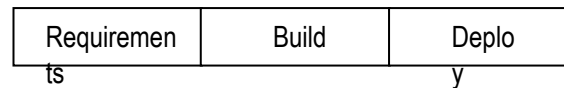
Timeboxing Execution

Linear

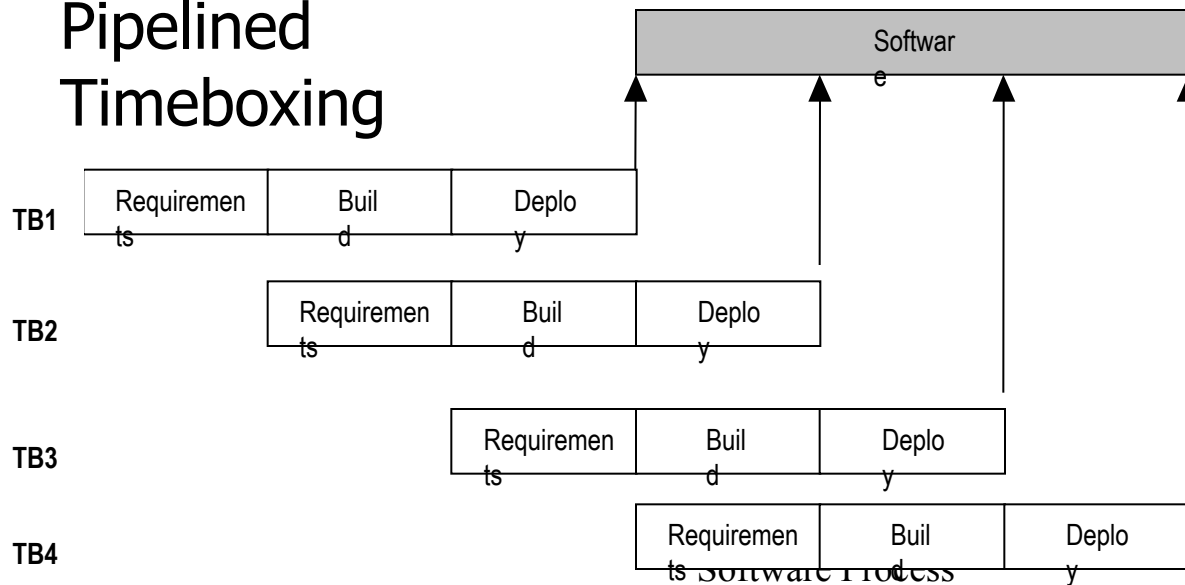
Timeboxing



TB2



Pipelined
Timeboxing



Time Boxed Iterations

- General iterative development – fix the functionality for each iteration, then plan and execute it
- In time boxed iterations – fix the duration of iteration and adjust the functionality to fit it
- Completion time is fixed, the functionality to be delivered is flexible

Linear Timeboxing

- This itself very useful in many situations
- Has predictable delivery times
- Overall product release and marketing can be better planned
- Makes time a non-negotiable parameter and helps focus attention on schedule
- Prevents requirements bloating
- Overall dev time is still unchanged

Pipelined Timeboxing

- Multiple iterations executing in parallel
- Can reduce the average completion time by exploiting parallelism
- For parallel execution, can borrow pipelining concepts from hardware
- This leads to Pipelined Timeboxing Process Model

Pipelined Timeboxing Model – Basics

- Development is done iteratively in fixed duration time boxes
- Each time box divided in fixed stages
- Each stage performs a clearly defined task that can be done independently
- Each stage approximately equal in duration
- There is a dedicated team for each stage
- When one stage team finishes, it hands over the project to the next team

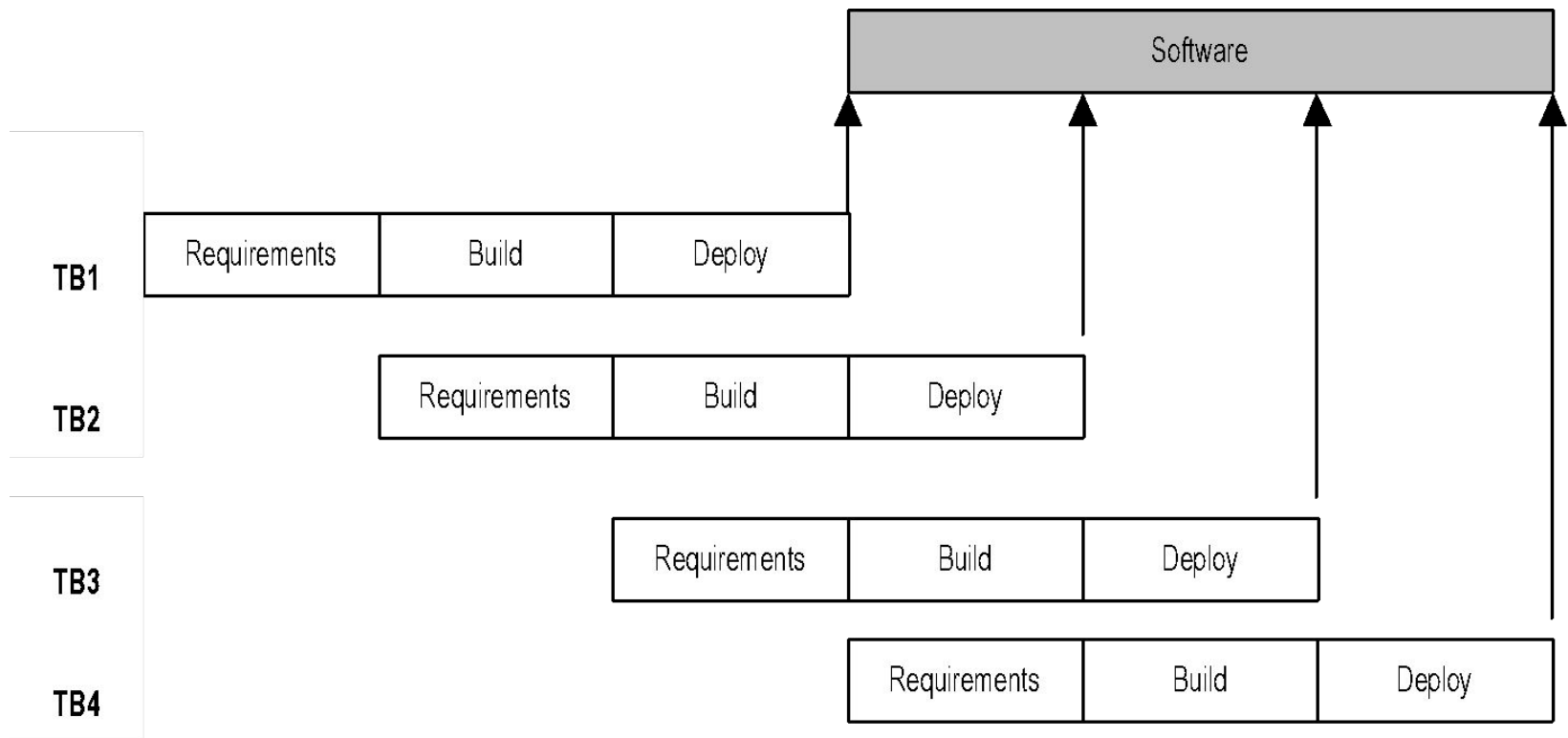
Example

- An iteration with three stages – Analysis, Build, Deploy
 - These stages are appx equal in many situations
 - Can adjust durations by determining the boudaries suitably
 - Can adjust duration by adjusting the team size for each stage
- Have separate teams for A, B, and D

Pipelined Execution

- A Team starts executing it-1
- A Team finishes, hands over it-1 to B Team, starts executing it-2
- A Team finishes it-2, hands over to B Team; B Team finishes it-1, hands over to D Team; AT starts it-3, B Team starts it-2 (and D Team, it-1)
- ...

Timeboxing Execution



Timeboxing execution

- Duration of each iteration still the same
- Total work done in a time box is also the same
- Productivity of a time box is same
- Yet, average cycle time or delivery time has reduced to a third
- Magic?
 - Are we getting something for nothing?

Team Size

- No magic, bigger teams
- In linear execution of iterations, the same team performs all stages
- If each stage has a team of size S , in linear execution the total team size is S
- In pipelined execution, the total team size is three times (one for each stage)
- Total team size in timeboxing is larger; and this what reduces cycle time

Brook's Law

- “Adding manpower to a late software project makes it later”.
 - Fred Brooks, The Mythical Man-Month (1975)

http://en.wikipedia.org/wiki/Brooks's_law

Team Size

- Merely by increasing the team size we cannot reduce cycle time - Brook's law
- Timeboxing allows structured way to add manpower to reduce cycle time
- Note that we cannot change the time of an iteration – Brook's law still holds
- Work allocation different to allow larger team to function properly

Work Allocation of Teams

Requirements Team	Requirements Analysis for TB1	Requirements Analysis for TB2	Requirements Analysis for TB3	Requirements Analysis for TB4	
Build Team		Build for TB1	Build for TB2	Build for TB3	Build for TB4
Deployment Team			Deployment for TB1	Deployment for TB2	Deployment for TB3

Timeboxing

- **Advantages:**
 - Shortened delivery times,
 - other adv of iterative,
 - distr. Execution
- **Disadvantages:**
 - Larger teams,
 - proj mgmt is harder,
 - high synchronization needed,
 - CM is harder
- **Applicability**
 - When short delivery times
 - When architecture is stable
 - Flexibility in feature grouping
 - For larger heavily managed teams

Summary

- Process is a means to achieve project objectives of high Q&P
- Process models define generic process, which can form basis of project process
- Process typically has stages, each stage focusing on an identifiable task
- Many models for development process have

Summary – waterfall

Strength	Weakness	Types of Projects
Simple Easy to execute Intuitive and logical Easy contractually	All or nothing – too risky Req frozen early May chose outdated hardware/tech Disallows changes No feedback from users Encourages req bloating	Well understood problems, short duration projects, automation of existing manual systems

Summary – Prototyping

Strength	Weakness	Types of Projects
Helps req elicitation Reduces risk Better and more stable final system	Front heavy Possibly higher cost and schedule Encourages req bloating Disallows later change	Systems with novice users; or areas with req uncertainty. Heavy reporting based systems can benefit from UI proto

Summary – Iterative

Strength	Weakness	Types of Projects
Regular deliveries, leading to biz benefit Can accommodate changes naturally Allows user feedback Avoids req bloating Naturally prioritizes req Allows reasonable exit points Reduces risks	Overhead of planning each iteration Total cost may increase System arch and design may suffer Rework may increase	For businesses where time is imp; risk of long projects cannot be taken; req not known and evolve with time

Summary – Timeboxing

Strength	Weakness	Types of Projects
All benefits of iterative Planning for iterations somewhat easier Very short delivery times	PM becomes more complex Team size is larger Complicated – lapses can lead to losses	Where very short delivery times are very important Where flexibility in grouping features Arch is stable